

EVALUACIÓN 1 · API DEL MUNDIAL

OBJETIVO

Qué vas a construir y qué se está evaluando.

OBJETIVO GENERAL

Construir el backend de "**Mundial 2026**": una API REST que administra las **selecciones** que participan en el torneo, organizadas por **grupos** y por **continentes**, y que permite registrar los **resultados de las semifinales y la final**; y **explicar tus propias decisiones técnicas** en un video demostrativo.

OBJETIVOS DE APRENDIZAJE

Al terminar esta evaluación, deberás ser capaz de:

1. **Diseñar y construir rutas con Express**, usando correctamente los parámetros de ruta (`:id`), los de consulta (`? continente=`) y el cuerpo de la petición.
2. **Resolver búsquedas y transformaciones sobre colecciones** con los métodos de array (`find` , `filter` , `map` , `flatMap`), incluyendo **búsquedas anidadas** entre entidades relacionadas.
3. **Modificar el estado de la aplicación** desde una ruta `POST` , registrando datos nuevos y validando lo que llega en el cuerpo de la petición.
4. **Responder con el código de estado HTTP correcto** según el resultado de cada operación, y manejar los casos de error sin que el servidor se caiga.
5. **Explicar y defender tus decisiones técnicas**: justificar por qué elegiste cada algoritmo, demostrar el funcionamiento en vivo, y **reconocer los límites de tu propia solución**.

Toda la información vive **en memoria**, en arrays dentro de tu programa.

SIN BASE DE DATOS

Esta evaluación **no usa MongoDB**. Los datos se guardan en **arrays** declarados en tu código. Si reinicias el servidor, los cambios se pierden: **es lo esperado**. Lo que se evalúa aquí es tu manejo de **rutas** y de **algoritmos de búsqueda y transformación**.

ALGUNOS MÉTODOS QUE SON PROPIOS DE JAVASCRIPT, QUE TE PERMITIRÁN RESOLVER LOS ALGORITMOS

`.find()`

`.filter()`

`.map()`

`.flatMap()`

`.push()`

`.some()`

`.sort()`

`.reduce()`

REPASO RAPIDO DE LAS HERRAMIENTAS

Ya los viste en unidades anteriores. Aquí van refrescados, con otros datos.

ESTOS EJEMPLOS NO SON DEL MUNDIAL

Son de una biblioteca, a propósito: te muestran **cómo funciona cada método**, no cómo resolver tu evaluación. La traducción al problema de las selecciones es parte de tu trabajo.

```
const libros = [
  { titulo: 'Rayuela', autorId: 1, generos: ['novela', 'experimental'], paginas: 736 },
  { titulo: 'Ficciones', autorId: 2, generos: ['cuento', 'fantástico'], paginas: 174 },
  { titulo: 'El Aleph', autorId: 2, generos: ['cuento'], paginas: 146 },
]
const autores = [
  { id: 1, nombre: 'Cortázar', pais: 'Argentina' },
  { id: 2, nombre: 'Borges', pais: 'Argentina' },
]
```

find — encontrar UNO

Devuelve **el primero** que cumpla, o `undefined`.

```
const b = libros.find(l => l.titulo === 'El Aleph')
// { titulo: 'El Aleph', autorId: 2, ... }

libros.find(l => l.titulo === 'Nada')
// undefined ← ojo: por eso valides antes de usarlo
```

filter — quedarte con VARIOS

Devuelve un **array** con todos los que cumplan. Si ninguno cumple, devuelve `[]` (no `undefined`).

```
libros.filter(l => l.paginas < 200)
// [ { Ficciones... }, { El Aleph... } ]

// ¿tiene al menos un género? el dato es un ARRAY:
libros.filter(l => l.generos.length > 0)
```

Búsqueda anidada — cuando el dato está en OTRO array

El libro no guarda el nombre del autor: guarda su `autorId`. Para buscar "los libros de Borges" hay que dar **dos saltos**: primero encontrar al autor por su nombre, después filtrar los libros por su id.

```
// ❌ No funciona: el libro no tiene la propiedad "autor"
libros.filter(l => l.autor === 'Borges')

// ✅ Dos pasos: primero el autor, después sus libros
const autor = autores.find(a => a.nombre === 'Borges') // 1) resolver la referencia
if (!autor) return [] // 2) ¿y si no existe?
const suyos = libros.filter(l => l.autorId === autor.id) // 3) ahora sí, filtrar
// [ { Ficciones... }, { El Aleph... } ]
```

🌀 flatMap — aplanar arrays que viven dentro de objetos

Cada libro tiene **su propio array** de géneros. Si usas `map`, te quedan **arrays dentro de un array**. `flatMap` hace el `map` y los aplanan en una sola lista.

```
libros.map(l => l.generos)
// [ ['novela','experimental'], ['cuento','fantástico'], ['cuento'] ] ← anidado 🤔

libros.flatMap(l => l.generos)
// [ 'novela', 'experimental', 'cuento', 'fantástico', 'cuento' ] ← plano 🎉
```

📊 reduce — resumir una lista en UN solo valor solo para /estadísticas · 2%

Recorre el array acumulando. Ese acumulador puede ser un número... o un **objeto**, que es lo que sirve para *contar por categoría*.

```
// Acumulador NÚMERO: total de páginas
const total = libros.reduce((acc, l) => acc + l.paginas, 0)
// 1056          ↑ acumulador          ↑ valor inicial

// Acumulador OBJETO: cuántos libros por autor
const porAutor = libros.reduce((acc, l) => {
  const nombre = autores.find(a => a.id === l.autorId).nombre // ← anidada adentro
  acc[nombre] = (acc[nombre] || 0) + 1 // si no existe la clave, empieza en 0
  return acc // ← SIEMPRE devolver el acumulador
}, {}) // ← ahora el inicial es un objeto vacío
// { 'Cortázar': 1, 'Borges': 2 }
```

El error más común: olvidar el `return acc`. Sin él, el acumulador se pierde en la vuelta siguiente. Si tu `reduce` devuelve `undefined`, empieza por ahí.

✓ UN ARRAY VACÍO NO ES UN ERROR

`filter` devuelve `[]` cuando nada cumple; `find` devuelve `undefined` cuando no encuentra. **No es lo mismo**, y tu API debe tratarlos distinto: una lista vacía suele ser un `200`; algo que no existe, un `404`.

REQUERIMIENTOS TECNICOS

Lo que tu API debe tener. Léelo completo antes de escribir código.

1. Los datos: te los damos hechos

✓ NO TIENES QUE INVENTAR NI TIPEAR LOS DATOS

Junto a este enunciado se te entrega el archivo `datos-mundial.js` con los tres arrays **ya cargados**: 16 selecciones repartidas en 4 grupos y 5 continentes, con sus copas reales. **Cópialo tal cual en tu proyecto**. Tu trabajo empieza en las rutas.

Esta es la forma de los datos que vas a recibir. Puedes agregar campos si los necesitas, pero **no quites los que están**:

```
// Un continente agrupa selecciones
export const continentes = [
  { id: 1, nombre: 'Sudamérica', confederacion: 'CONMEBOL' },
  { id: 2, nombre: 'Europa', confederacion: 'UEFA' },
  // ... (5 en total)
]

// Los grupos del torneo
export const grupos = [
  { id: 1, nombre: 'A' },
  // ... (4 en total)
]

// Cada selección pertenece a UN grupo y a UN continente (por su id).
// `copas` = los años de los mundiales que ganó.
export const selecciones = [
  { id: 1, nombre: 'Brasil', groupId: 1, continenteId: 1, fifaRanking: 5, copas: [1958, 1962, 1970, 1994, 2002] },
  { id: 2, nombre: 'Chile', groupId: 1, continenteId: 1, fifaRanking: 45, copas: [] },
  // ... (16 en total)
]

// Aquí se guardan los resultados que registres. Empiezan vacíos:
// los llenan tus rutas POST.
export const partidos = {
  semifinales: [], // { numero: 1..4, local: {...}, visita: {...} }
  final: null,
}
```

📌 FÍJATE EN CÓMO SE RELACIONAN

La selección **no guarda** el nombre de su continente ni el de su grupo: guarda su `id`. Para responder "las selecciones de Europa" tienes que **resolver esa referencia**. Ese salto entre arrays es una **búsqueda anidada**, y es justo lo que se está evaluando.

2. Las rutas base (rápidas)

Sin estas no hay API que probar. Son directas: te sirven para levantar Express y verificar que todo responde.

RUTA	QUÉ DEBE HACER
<code>GET /api/selecciones</code>	Devuelve todas las selecciones.
<code>GET /api/selecciones/:id</code>	Devuelve una selección por su id. Si no existe → <code>404</code> .

3. Las rutas con lógica ★ (el corazón de la evaluación)

Aquí está lo que de verdad se evalúa. Cada una te obliga a resolver un problema distinto.

RUTA	QUÉ DEBE HACER
<code>GET /api/selecciones?continente=Europa</code>	 Búsqueda anidada. La selección solo tiene <code>continenteId</code> , no el nombre. Tienes que buscar primero el continente por su nombre y después filtrar las selecciones por su id.
<code>GET /api/selecciones?campeon=true</code>	 Filtro. Solo las selecciones que ganaron al menos una copa . Piensa: ¿cómo distingues una que ganó de una que no, si el dato es un array?
<code>GET /api/copas</code>	 Aplanar. El listado plano de todas las copas de todas las selecciones. Cada selección tiene su propio array de copas: hay que unirlos todos en una sola lista. <i>Pista:</i> <code>flatMap()</code> .
<code>GET /api/copas/:seleccion</code>	 Buscar por nombre. Las copas de una selección, buscándola por su nombre (ej. <code>/api/copas/argentina</code> → <code>[1978, 1986, 2022]</code>). Devuelve el arreglo de años . Si la selección no existe → <code>404</code> .
<code>GET /api/estadisticas</code> 2%	 Resumir · el más exigente. Un resumen del torneo en un solo objeto: total de selecciones, total de copas repartidas, cuántas selecciones hay por continente , y el ranking FIFA promedio . <i>Pista:</i> <code>reduce()</code> . Vale solo 2% de la nota: si no te sale, casi no te afecta. Está para quien quiera ir más allá.

⚠ OJO: "NO EXISTE" Y "NO TIENE" SON COSAS DISTINTAS

`/api/copas/chile` → Chile **existe**, solo que no ha ganado ninguna copa: responde `200` con `[]`.

`/api/copas/narnia` → esa selección **no existe**: responde `404`.

Un arreglo vacío **no es un error**. El `404` significa "el recurso no existe", no "está vacío".

🔍 BUSCAR POR NOMBRE, NO POR ID

El usuario escribe `/api/copas/argentina`, no `/api/copas/5`. Piensa cómo comparar: ¿qué pasa si escriben `Argentina`, `argentina` o `ARGENTINA`? Tu API debería responder igual en los tres casos.

4. Semifinales y final ★

El torneo tiene **cuatro semifinales** y **una final**. Cada una registra el resultado con los goles de las dos selecciones que se enfrentan.

✅ NO SON CUATRO RUTAS: ES UNA SOLA

`/semifinals/1`, `/semifinals/2`, `/semifinals/3` y `/semifinals/4` se resuelven con **una única ruta parametrizada**: `/semifinals/:n`. El número llega como parámetro. Si escribiste cuatro rutas casi idénticas, vuelve a mirarla.

RUTA	QUÉ DEBE HACER
<code>POST /api/worldcup/2026/semifinals/:n</code>	Registra el resultado de la semifinal <code>n</code> (de 1 a 4). Recibe las dos selecciones y sus goles. Si <code>n</code> no está entre 1 y 4 → <code>400</code> .
<code>GET /api/worldcup/2026/semifinals/:n</code>	Devuelve el resultado de esa semifinal. Si aún no se jugó → <code>404</code> .
<code>GET /api/worldcup/2026/semifinals</code>	Las cuatro semifinales, con su resultado (o vacías si no se jugaron).
<code>POST /api/worldcup/2026/final</code>	Registra el resultado de la final, igual que una semifinal.
<code>GET /api/worldcup/2026/final</code>	Devuelve la final: quién jugó, los goles, y quién ganó .

FORMA DEL CUERPO (POST DE UN PARTIDO)

```
{
  "local":    { "seleccionId": 1, "goles": 2 },
  "visita":   { "seleccionId": 7, "goles": 1 }
}
```

FORMA DE LA RESPUESTA (GET DE UN PARTIDO)

```
{
  "partido": "semifinal 1",
  "local":   { "seleccion": "Chile",    "goles": 2 },
  "visita":  { "seleccion": "Alemania", "goles": 1 },
}
```

```
"ganador": "Chile"
}
```

📌 FÍJATE EN LA RESPUESTA

El `POST` recibe `ids`, pero el `GET` responde con `nombres`. Tienes que resolver cada id a su selección: otra búsqueda anidada. Y `ganador` se **calcula** comparando los goles — no se guarda.

5. Manejo de errores y códigos de estado

- Usa el **código correcto** en cada caso: `200` al leer bien, `201` al crear, `400` si los datos que llegan están mal o faltan, `404` si el recurso no existe.
- Nunca respondas `200` con un error adentro del cuerpo.
- Los errores deben devolver un mensaje claro, ej: `{ "error": "No existe la selección 99" }`.
- Una ruta que no existe debe responder `404`, no reventar el servidor.

☀️ Desafíos extra (opcionales)

- **Tabla de posiciones:** `GET /api/grupos/:nombre/tabla` — las selecciones del grupo ordenadas por su ranking FIFA (usa `sort()`).
- **Camino al título:** `GET /api/worldcup/2026/camino/:seleccionId` — todos los partidos que jugó una selección (semifinal y final), con su resultado en cada uno.
- **Al campeón, su copa:** que al registrar la final, el ganador reciba automáticamente la copa `2026` en su array de `copas`.

ENTREGABLES

Qué tienes que entregar, y cómo.

1. Un repositorio público en GitHub

Con tu código. Debe incluir un `.gitignore` con, al menos:

```
node_modules
.env
```

⚠ NUNCA SUBAS NODE_MODULES

Si tu repositorio pesa cientos de megas, es que subiste `node_modules`. Esa carpeta se regenera con `npm install`: no se versiona.

2. Un `README.md` que explique cómo levantar tu proyecto

Breve. Qué instalar, qué comando corre el servidor, y en qué puerto queda escuchando. Incluye también el enlace de YouTube a tu video (ver punto 4).

3. Una colección de Postman exportada (`.json`)

Dentro del repositorio. No hace falta que incluya todas las variantes: con **una petición por cada ruta** alcanza, y las de `POST` con su cuerpo de ejemplo ya cargado, listas para ejecutar.

✅ CÓMO SE VA A PROBAR

El profesor va a clonar tu repo, correr `npm install`, levantar tu servidor y ejecutar tu colección de Postman. Si eso no funciona, no se puede evaluar tu trabajo. Pruébalo tú antes de entregar: clona tu propio repositorio en otra carpeta y verifica que arranca desde cero.

4. Un video demostrativo (15% de la nota)

Duración: mínimo 7 minutos, máximo 15.

Súbelo a YouTube y deja el enlace en tu `README.md`. Puedes publicarlo como "no listado": así solo lo ve quien tenga el enlace, no aparece en búsquedas ni en tu canal. Verifica que el enlace **se abra sin pedir permisos** — pruébalo en una ventana de incógnito antes de entregar.

📌 QUÉ SE EVALÚA AQUÍ

No es narrar tu código línea por línea. Es **demostrar que entiendes lo que construiste**: por qué elegiste cada algoritmo, qué pasa cuando algo se rompe, y hasta dónde aguanta tu solución. Se te va a ver **trabajando en vivo**, no leyendo un guion.

CONTENIDO OBLIGATORIO DEL VIDEO

1. **Tu API funcionando (Postman).** Muestra las rutas principales respondiendo de verdad: una consulta, un filtro por continente, el `POST` de una copa y el registro de una semifinal. Que se vean los **códigos de estado** en pantalla.

2. **Tus algoritmos, explicados por ti.** Abre tu código y explica **con tus palabras** las decisiones que tomaste. No describas la sintaxis: justifica el *por qué*.

- ¿Por qué `flatMap` y no un `map` seguido de otra cosa?
- ¿Cómo resolviste la **búsqueda anidada** del continente?
- ¿Cómo decides si una selección es campeona, si el dato es un *array* de copas?
- Si resolviste `/estadísticas`: ¿qué hace tu `reduce`, paso a paso?

3. **Modificación del código en vivo — CORS.** Esta parte es obligatoria. Se hace **sobre tu código**, en tu editor: no necesitas navegador ni Postman aquí.

1. **Muestra dónde configuraste CORS** en tu proyecto y explica, con tus palabras, **qué hace esa configuración**.
2. **Modifícala en vivo:** restringe los verbos permitidos para que `DELETE` quede fuera.
3. **Explica qué pasaría** con esa configuración: quién bloquearía la petición, en qué momento, y por qué el servidor igual la recibió. *Puedes mostrarlo si quieres, pero no es obligatorio: lo que se evalúa es tu explicación.*
4. **Deja tu código como estaba.**

🔴 AQUÍ SE EVALÚA QUE EXPLIQUES, NO QUE EJECUTES

Lo importante no es ver el error en pantalla: es que puedas **razonar sobre tu propia configuración** y anticipar qué provoca cambiarla. Tómate tu tiempo en esta parte — es donde mejor se nota que entiendes lo que escribiste.

4. **Los límites de tu solución (escalabilidad).** Cierra explicando, con tus palabras, **qué problemas tiene este diseño si el proyecto creciera**. Piensa en:

- Los datos viven **en memoria**: ¿qué pasa cuando el servidor se reinicia? ¿y si hubiera dos servidores atendiendo?
- Las búsquedas recorren el array **entero** cada vez: ¿qué pasa con 32 selecciones? ¿y con 100.000 registros?
- Las relaciones se resuelven **a mano** (buscar el continente, después filtrar): ¿qué se complica al agregar más entidades?
- ¿Qué cambiarías tú si tuvieras que llevar esto a producción?

✅ AQUÍ NO HAY RESPUESTA ÚNICA

Esta última parte **no tiene una solución correcta**: se evalúa que **identifiques límites reales** de tu propio trabajo y los argumentos. Reconocer lo que tu código todavía no resuelve es una habilidad profesional, no una debilidad.

REQUISITOS TÉCNICOS DEL VIDEO

- **Duración:** entre **7 y 15 minutos**. Menos de 7 no alcanza a cubrir lo pedido; más de 15 no se revisa completo.
- Tu **voz** explicando. El código y Postman deben **leerse** en pantalla (sube el tamaño de la fuente del editor).
- Cámara opcional. Lo que importa es la pantalla y tu explicación.
- No hace falta editarlo ni que sea perfecto: **una toma continua está bien**. Si te equivocas y lo resuelves en vivo, mejor todavía.

PONDERACION

Cómo se reparte la nota.

35% **Funcionalidad de las rutas**
 Todos los endpoints existen y hacen lo que prometen. Las semifinales y la final registran y devuelven sus resultados correctamente.

28% **Manejo de datos y algoritmos**
 Uso correcto y pertinente de `find`, `filter` y `flatMap`. Las búsquedas anidadas resuelven bien las referencias entre selecciones, grupos y continentes.

12% **Códigos de estado y errores**
 Cada respuesta usa el status correcto (200 / 201 / 400 / 404). Los errores traen un mensaje claro y el servidor no se cae ante datos inválidos.

8% **Entrega y orden**
 El repositorio levanta siguiendo el README. La colección de Postman está completa. El código está ordenado y el `.gitignore` es correcto.

2%  **GET /api/estadisticas – el reduce**
 Vale poco a propósito: es la ruta más exigente y está para quien quiera ir más allá. Si no te sale, casi no afecta tu nota.

15%  **Video demostrativo**
 Demostración en vivo, explicación de tus algoritmos, la demo de CORS, y el análisis de escalabilidad. Rúbrica detallada más abajo.

SOBRE LOS DESAFÍOS EXTRA

Son **opcionales** y no restan si no los haces. Se consideran para subir la nota cuando el resto está bien resuelto.

RUBRICA A · LA API (85%)

Cada criterio se evalúa en uno de estos cuatro niveles.

CRITERIO	%	4 · DESTACADO	3 · LOGRADO	2 · EN DESARROLLO	1 · INSUFICIENTE
Funcionalidad de las rutas Los endpoints hacen lo que prometen	35	Todas las rutas funcionan, incluidas las 4 semifinales y la final. Los filtros por query (<code>? continente</code> , <code>? campeon</code>) responden bien. Resuelve al menos un desafío extra.	Todas las rutas obligatorias funcionan. Puede haber fallas menores en algún filtro o caso borde.	Las rutas básicas funcionan, pero faltan algunas (semifinales incompletas, o los filtros por query no operan).	Faltan rutas centrales o el servidor no responde correctamente.
Manejo de datos y algoritmos find · filter · flatMap	28	Usa el método adecuado a cada problema y lo justifica. El <code>flatMap</code> de copas está bien resuelto. Las búsquedas anidadas son limpias, sin recorridos innecesarios.	Usa correctamente los métodos pedidos. Las búsquedas anidadas funcionan, aunque alguna sea más larga de lo necesario.	Resuelve con métodos de array, pero elige mal alguno (ej. <code>forEach</code> + <code>push</code> donde correspondía <code>map</code>) o las búsquedas anidadas fallan en algún caso.	Usa ciclos <code>for</code> en vez de los métodos pedidos, o las búsquedas anidadas no funcionan.
Códigos de estado y errores 200 · 201 · 400 · 404	12	Cada respuesta usa el status exacto. Valida la entrada, responde mensajes de error claros y útiles, y el servidor nunca se cae.	Usa los status correctos en los casos principales. Los errores traen mensaje.	Usa algunos status correctos, pero confunde otros (ej. <code>200</code> al crear, o <code>500</code> donde va <code>404</code>).	Responde siempre <code>200</code> , o el servidor se cae con datos inválidos.
Entrega y orden Repo · README · Postman	8	El repo levanta siguiendo el README, sin fricción. La colección de Postman está completa y con ejemplos. Código ordenado y	El repo levanta. README y colección de Postman presentes y utilizables.	El repo levanta con dificultad (falta un paso en el README, o la colección está incompleta).	No levanta, falta el README o la colección de Postman, o subió <code>node_modules</code> .

		<code>.gitignore</code> correcto.			
 /estadísticas El reduce · para ir más allá	2	El resumen está completo y correcto (totales, agrupación por continente y promedio), con <code>reduce</code> bien usado.	La ruta responde con el resumen, aunque falte algún dato o use un camino más largo.	La ruta existe pero el resumen está incompleto o mal calculado.	No la implementó. Es lo esperado si no llegó: vale 2%.

RUBRICA B · EL VIDEO (15%)

Duración: 7 a 15 minutos. Se evalúa que demuestres dominio, no que narres tu código.

CRITERIO	%	4 · DESTACADO	3 · LOGRADO	2 · EN DESARROLLO	1 · INSUFICIENTE
Demostración funcional La API andando en Postman	3	Recorre las rutas con fluidez, mostrando los status. Elige buenos casos, incluidos errores (404 / 400) para demostrar que los maneja.	Muestra las rutas principales funcionando y se leen los status.	Muestra algunas rutas, pero salta partes pedidas o no se alcanza a leer la pantalla.	No demuestra la API funcionando, o no se ve lo que ocurre.
Explicación de tus algoritmos El <i>por qué</i> , no el <i>qué</i>	4	Justifica cada elección: por qué <code>flatMap</code> y no otra cosa, cómo resolvió la búsqueda anidada, cómo distingue a las campeonas. Se nota que el código es suyo y que entiende cada decisión.	Explica sus algoritmos con sus palabras y se entiende. Justifica las decisiones principales.	Describe el código ("aquí hago un filter...") pero sin justificar por qué. Suen a lectura.	No explica sus algoritmos, o la explicación no corresponde con el código entregado.
Modificación en vivo · CORS Sobre tu código, en tu editor	5	Ubica su configuración de CORS y explica qué hace. La modifica en vivo dejando <code>DELETE</code> fuera y anticipa con precisión qué ocurriría : quién bloquea, en qué momento, y por qué el servidor igual recibió la petición. Restaura su código.	Modifica la configuración y explica correctamente qué provocaría el cambio, aunque de forma breve. Restaura su código.	Modifica el código, pero la explicación de qué pasaría es vaga o imprecisa, o no lo deja como estaba.	No modifica su configuración en vivo, o no logra explicar qué hace ni qué provocaría cambiarla.

Análisis de escalabilidad Los límites de tu propia solución	3	Identifica límites reales y concretos (datos en memoria que se pierden, búsquedas $O(n)$ que no escalan, relaciones resueltas a mano) y propone con criterio qué cambiaría para producción.	Identifica al menos dos límites reales del diseño y los argumenta.	Menciona limitaciones de forma vaga o genérica ("habría que usar una base de datos"), sin conectarlas con su código.	No aborda la escalabilidad.
Formato y duración 7 a 15 minutos	—	Requisito, no puntaje. Un video de menos de 7 minutos o de más de 15 se evalúa solo hasta donde cumple . Si el código no se lee en pantalla, ese criterio no se puede evaluar. Sube el video a YouTube (puede ser <i>no listado</i>) y deja el enlace en tu <code>README.md</code> . El enlace debe abrirse sin pedir permisos.			
Total del video	15	El video no reemplaza al código: lo demuestra. Un video excelente sobre una API que no funciona no compensa la Rúbrica A.			

EL VIDEO ES INDIVIDUAL Y ES TUYO

Se evalúa **tu** comprensión de **tu** código: por eso lo que más suma es que **justifiques tus propias decisiones**, no que la solución sea perfecta. Explicar por qué elegiste un camino —y qué no te terminó de convencer— vale más que recitar lo que hace cada línea.